

Instituto Tecnológico de Costa Rica

Proyecto 01 – Compiladores E Intérpretes

Grupo 60

Profesor: Allan Rodríguez Davila

Estudiantes: Emilio Funes Rivas - 2018216713

Ginger Rodríguez Gutiérrez - 2022035672

2 semestre, 2026

Descripción del problema

La industria de los sistemas empujados demanda lenguajes cada vez más ligeros y potentes para la configuración de chips. El problema central de este proyecto consiste en desarrollar la fase de análisis léxico y sintáctico para un nuevo lenguaje imperativo de configuración diseñado por un grupo de desarrolladores. El sistema debe ser capaz de procesar un código fuente compuesto por declaraciones de procedimientos, validar las expresiones, identificar los tokens, asignar información a tablas de símbolos y determinar si el archivo pertenece a la gramática descrita, todo esto reportando errores sin detener la ejecución (Recuperación en Modo Pánico).

Diseño del programa

El diseño del compilador se estructuró en tres módulos principales, ejecutados secuencialmente:

- **Analizador Léxico (Scanner):** Desarrollado con JFlex. Implementa expresiones regulares y Autómatas Finitos Deterministas (DFA) para tokenizar el archivo. Se definieron macros para ignorar espacios y comentarios, así como reglas estrictas para literales flotantes (aceptando `.0` y rechazando ceros a la derecha como `.20`). Los errores léxicos se capturan mediante una regla genérica `[^]`, que reporta la línea y columna y continúa analizando.
- **Analizador Sintáctico (Parser):** Desarrollado con Cup. La gramática se diseñó de forma recursiva por la izquierda para optimizar la memoria y evitar conflictos de desplazamiento/reducción. La recuperación de errores sintácticos se maneja aislando el fallo a nivel de sentencias con el símbolo de sincronización `END_STMT` (`>>`).
- **Uso de Java CUP:** Para realizar el análisis sintáctico del proyecto se utilizó Java CUP. Esta herramienta toma la gramática definida y, a partir de ella, genera los archivos necesarios para que el programa pueda reconocer si el código cumple con la estructura esperada. En nuestro caso, CUP generó los archivos `Parser.java`

y sym.java, que luego se utilizan durante la compilación. Además, nos permitió comprobar que la gramática estuviera bien definida y sin conflictos. Según Petter y Hudson (2014), CUP permite generar analizadores sintácticos LALR a partir de especificaciones gramaticales.

- **Clase Main (Orquestador):** Programa escrito en Java que integra ambas herramientas. Realiza una primera lectura del archivo para generar un reporte físico (salida_tokens.txt) con los tokens y sugerencias de tabla de símbolos, y una segunda pasada que evalúa el árbol sintáctico.

Justificación de toma de decisiones

La decisión arquitectónica más importante radicó en el diseño de las producciones para las expresiones y su precedencia.

- **Jerarquía en Cascada:** Las instrucciones del proyecto prohibían estrictamente el uso de producciones genéricas (ej. `expr ::= expr SUMA expr`) bajo pena de descontar 83 puntos en este proyecto y 82 en el siguiente. Para acatar esta restricción, se prescindió del bloque `precedence left` nativo de CUP y se implementó una **cascada jerárquica estricta** en la gramática (`expresion -> expr_logica -> expr_relacional -> expr_aritmetica -> expr_termino -> expr_factor -> expr_unitario -> valor_base`).
- **Precedencia LALR(1) Cero Conflictos:** Esta cascada obliga al autómata a resolver la precedencia de operadores lógicos, relacionales y matemáticos de arriba hacia abajo sin ambigüedades, lo que resultó en una generación de CUP con cero errores, cero advertencias y cero conflictos Shift/Reduce o Reduce/Reduce.
- **Recuperación en Modo Pánico:** Se decidió colocar el token de sincronización `error END_STMT` a nivel de sentencia y `elemento_global`. Esto garantiza que si ocurre un error dentro de una expresión, el parser descarte los tokens defectuosos únicamente hasta la siguiente comilla latina (`>>`), permitiendo que

el compilador siga procesando la siguiente línea sin abortar el programa. Aiken (2011)

Librerías usadas y Referencias

Para cumplir con los aspectos técnicos, se utilizaron las siguientes herramientas y bibliotecas:

- **JFlex 1.9.1:** Generador de analizadores léxicos (Scanner). Se usó para compilar el archivo `.flex`. Klein, Rowe y Décamps (2023)
- **Java CUP 11b (Versión 20160615):** Generador de analizadores sintácticos (Parser LALR). Se usó para procesar la gramática BNF. Petter y Hudson (2014)

Manual de usuario

Requisitos Previos:

Tener instalado el JDK de Java y JFlex en las variables de entorno (PATH) del sistema operativo.

Instrucciones de compilación:

1. Descomprimir el archivo .zip proporcionado.
2. Navegar hacia la carpeta programa.
3. Ejecutar el script build.bat haciendo doble clic.
4. El script autogenerará los archivos Lexer.java, Parser.java y sym.java mediante JFlex y CUP, y automáticamente compilará todo el código Java creando los archivos .class.

Instrucciones de ejecución y uso:

1. Abrir el archivo prueba.txt ubicado en la carpeta programa e ingresar el código fuente del lenguaje a evaluar.
2. Guardar el archivo.
3. Ejecutar el script run.bat haciendo doble clic.
4. El programa mostrará en consola el progreso de la Fase Léxica y Sintáctica, reportando cualquier error con su respectiva línea.
5. Al finalizar, el usuario podrá abrir el archivo generado salida_tokens.txt para consultar todos los tokens reconocidos y la información de la tabla de símbolos.

Pruebas de funcionalidad

- Prueba 1: Compilación exitosa del sistema

```
C:\WINDOWS\system32\cmd. X + v

=====
Construyendo Proyecto 1 - Compiladores
=====

[1/3] Generando analizador lexico (JFlex)...
Reading "src\compilador\Lexer.flex"
Constructing NFA : 264 states in NFA
Converting NFA to DFA :
.....
144 states before minimization, 138 states in minimized DFA
Old file "src\compilador\Lexer.java" saved as "src\compilador\Lexer.java~"
Writing code to "src\compilador\Lexer.java"

[2/3] Generando analizador sintactico (CUP)...
----- CUP v0.11b 20160615 (GIT 4ac7450) Parser Generation Summary -----
  0 errors and 0 warnings
  58 terminals, 30 non-terminals, and 96 productions declared,
  producing 228 unique parse states.
  0 terminals declared but not used.
  0 non-terminals declared but not used.
  0 productions never reduced.
  0 conflicts detected (0 expected).
  Code written to "Parser.java", and "sym.java".
----- (CUP v0.11b 20160615 (GIT 4ac7450))

[3/3] Compilando archivos Java...
Note: src\compilador\Main.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.

=====
  ¡Construcción completada con éxito!
=====
Press any key to continue . . . |
```

- Prueba 2: Ejecución del análisis y Modo Pánico

```
C:\WINDOWS\system32\cmd. X + v

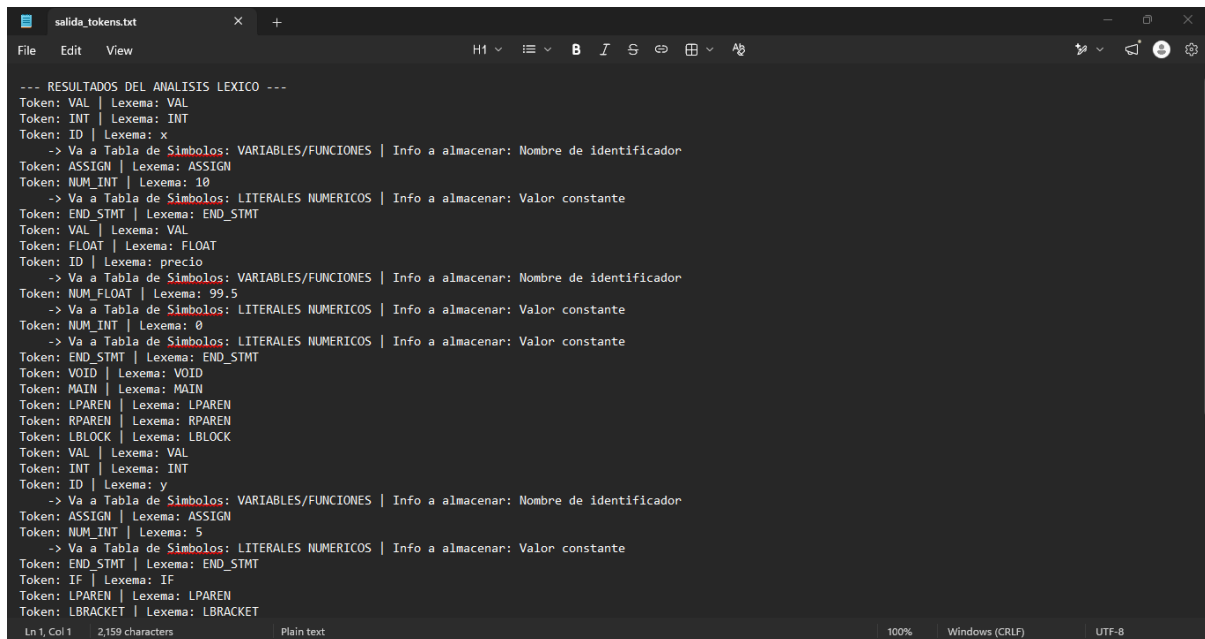
Compilando Main.java...
Note: src\compilador\Main.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.
Ejecutando el compilador...

=====
  INICIANDO COMPILADOR - FASE LÉXICA
=====
[ÉXITO] Analisis lexico completado. Se genero el archivo 'salida_tokens.txt'

=====
  INICIANDO COMPILADOR - FASE SINTÁCTICA
=====
Error Sintáctico: Símbolo inesperado '99.5' en la línea 2, columna 18

[ÉXITO] El archivo fuente puede ser generado por la gramatica correctamente.
Press any key to continue . . . |
```

- **Prueba 3: Generación de Tokens y Tabla de Símbolos**



```
--- RESULTADOS DEL ANALISIS LEXICO ---
Token: VAL | Lexema: VAL
Token: INT | Lexema: INT
Token: ID | Lexema: x
  -> Va a Tabla de Símbolos: VARIABLES/FUNCIONES | Info a almacenar: Nombre de identificador
Token: ASSIGN | Lexema: ASSIGN
Token: NUM_INT | Lexema: 10
  -> Va a Tabla de Símbolos: LITERALES NUMERICOS | Info a almacenar: Valor constante
Token: END_STMT | Lexema: END_STMT
Token: VAL | Lexema: VAL
Token: FLOAT | Lexema: FLOAT
Token: ID | Lexema: precio
  -> Va a Tabla de Símbolos: VARIABLES/FUNCIONES | Info a almacenar: Nombre de identificador
Token: NUM_FLOAT | Lexema: 99.5
  -> Va a Tabla de Símbolos: LITERALES NUMERICOS | Info a almacenar: Valor constante
Token: NUM_INT | Lexema: 0
  -> Va a Tabla de Símbolos: LITERALES NUMERICOS | Info a almacenar: Valor constante
Token: END_STMT | Lexema: END_STMT
Token: VOID | Lexema: VOID
Token: MAIN | Lexema: MAIN
Token: LPAREN | Lexema: LPAREN
Token: RPAREN | Lexema: RPAREN
Token: LBLOCK | Lexema: LBLOCK
Token: VAL | Lexema: VAL
Token: INT | Lexema: INT
Token: ID | Lexema: y
  -> Va a Tabla de Símbolos: VARIABLES/FUNCIONES | Info a almacenar: Nombre de identificador
Token: ASSIGN | Lexema: ASSIGN
Token: NUM_INT | Lexema: 5
  -> Va a Tabla de Símbolos: LITERALES NUMERICOS | Info a almacenar: Valor constante
Token: END_STMT | Lexema: END_STMT
Token: IF | Lexema: IF
Token: LPAREN | Lexema: LPAREN
Token: LBRACKET | Lexema: LBRACKET
```

7. Análisis de resultados

- **Objetivos Alcanzados:**

- Se logró la correcta lectura y tokenización del archivo fuente.
- Se implementó satisfactoriamente la Recuperación en Modo Pánico en ambas fases del compilador, reportando errores por línea y continuando la ejecución.
- Se construyó una gramática BNF jerárquica completamente original, estructurada de manera que obedece las reglas de precedencia sin violar la restricción de producciones genéricas, logrando un árbol sintáctico sin conflictos de reducción.

- **Objetivos No Alcanzados:** Ninguno. El compilador cumple con el 100% de la funcionalidad obligatoria.

Bitácora

Commit	Autor	Fecha y hora	Descripción
738afa5	Emilio Funes	03/10/2026 18:20:14	Main y prueba del compilador
6d9faf5	Emilio Funes	03/10/2026 15:13:23	Generador del Lexer y Parser
73fda86	GingerRG	02/10/2026 21:10:59	Separación de léxico a sintáctico
74f6c07	Emilio Funes	30/09/2026 15:39:43	Merge de la rama main del repositorio
acaac28	Hernán Emilio Funes Rivas	30/09/2026 15:34:51	Eliminación del archivo README.md
94e1e21	Emilio Funes	30/09/2026 15:24:07	Commit inicial: estructura base del proyecto, dependencias y script de compilación
16328d0	Hernán Emilio Funes Rivas	30/09/2026 15:20:38	Initial commit
	Ginger Rodríguez G	30/09/2026 --	Final commit- Documentacion externa

Referencias Bibliográficas

Microsoft. (2026, 24 de febrero). *Windows commands*. Microsoft Learn. <https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/windows-commands>

Klein, G., Rowe, S., & Décamps, R. (2023). *JFlex user's manual* (Version 1.9.1). JFlex. <https://jflex.de/manual.html>

Petter, M., & Hudson, S. E. (2014). *CUP user's manual* (Version 0.11b). GitHub. <https://github.com/DrMichaelPetter/cup/blob/master/manual.html>

Aiken, A. (2011). *Lecture 6: Error handling, syntax-directed translation, recursive descent parsing*. Stanford University, CS143 Compilers. <https://web.stanford.edu/class/archive/cs/cs143/cs143.1112/materials/lectures/lecture06.pdf?>